МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №4
з курсу «Професійний Python»

«Професійне ООП та система типів Python»

Виконав:
студент 3 курсу, групи ФЕП-31с
Гарбіч Данило

Перевірив:
професор
Свелеба Сергій Андрійович

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №4. «Професійне ООП та система типів Python».

3. Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої архітектури мовою Python із використанням класів, композиції, наслідування, абстрактних базових класів (`ABC`), структурних інтерфейсів (`Protocol`), датакласів (`dataclass`), інкапсуляції через властивості (`properties`), спеціальних методів (`dunder methods`), принципів SOLID та сучасної статичної системи типізації (`type hints`, `Generic`, `TypeVar`, `TypedDict`). Навчитися верифікувати кодову базу за допомогою статичного аналізатора типів `mypy`.

4. Номер індивідуального варіанта

Варіант №9: «Система обліку спортивних результатів».

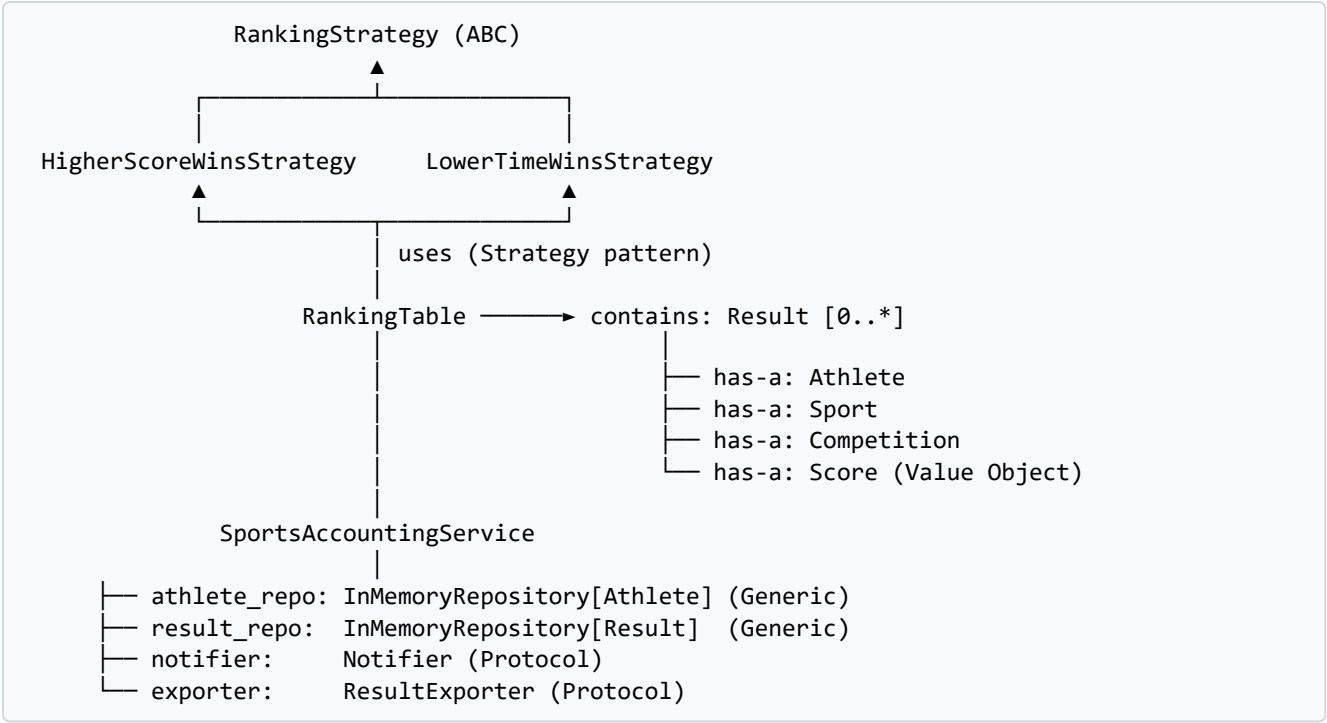
5. Опис предметної області

Предметна область охоплює облік та аналіз досягнень спортсменів на змаганнях різного рівня. Система забезпечує реєстрацію спортсменів, видів спорту, турнірів та фіксацію індивідуальних результатів. Змагальні результати мають різні метрики (бали, метри або секунди) та різні критерії лідерства (більший бал краще або менший час краще). Система формує рейтингові таблиці, розраховує агреговані статистичні показники та сповіщає зовнішні підсистеми про нові рекорди.

6. Список domain entities

- **`Athlete`** — сутність спортсмена (ID, ім'я, країна, вік);
- **`Sport`** — сутність спортивної дисципліни (ID, назва, одиниця вимірювання, прапорець виграшу: більше чи менше);
- **`Competition`** — сутність змагання (ID, назва, місто/локація, рік);
- **`Result`** — центральна сутність спортивного результату (композиція Athlete, Sport, Competition, Score);
- **`RankingTable`** — контейнер рейтингу спортсменів зі стратегією сортування та dunder-методами.

7. Class diagram



8. Пояснення inheritance

Наслідування використано для реалізації патерну **Strategy**: базовий абстрактний клас `RankingStrategy(ABC)` визначає інтерфейс методу `sort_results()`, а підкласи `HigherScoreWinsStrategy` та `LowerTimeWinsStrategy` реалізують специфічні правила сортування. Це відповідає відношенню **is-a** та принципу Liskov Substitution Principle (LSP).

9. Пояснення composition

Композиція реалізує зв'язок **has-a**: сутність `Result` компонується з `Athlete`, `Sport`, `Competition` та `Score`; `RankingTable` містить список результатів та стратегію ранжування; а сервіс `SportsAccountingService` об'єднує репозиторії та протоколи сповіщень. Це забезпечує слабку зв'язаність компонентів (low coupling) та максимальну гнучкість системи.

10. Приклад encapsulation

```
class Result:
    _score: Score

    @property
    def score(self) -> Score:
        return self._score

    @score.setter
    def score(self, new_score: Score) -> None:
        if new_score.value < 0:
            raise ValueError("Result score cannot be negative.")
        self._score = new_score
```

11. Приклад property

```
@property
def score(self) -> Score:
    return self._score
```

12. Приклад dataclass

```
@dataclass
class Athlete:
    id: int
    name: str
    country: str = "Україна"
    age: int = 20

    def __post_init__(self) -> None:
        self.name = self.name.strip()
        if not self.name:
            raise ValueError("Athlete name cannot be empty.")
```

13. Приклад immutable value object

```
@dataclass(frozen=True, slots=True)
class Score:
    value: float
    unit: str = "pts"

    def __post_init__(self) -> None:
        if self.value < 0:
            raise ValueError(f"Score value cannot be negative: {self.value}")

    def __add__(self, other: "Score") -> "Score":
        if self.unit != other.unit:
            raise ValueError(f"Unit mismatch: {self.unit} != {other.unit}")
        return Score(round(self.value + other.value, 4), self.unit)
```

14. Приклад ABC (Abstract Base Class)

```
from abc import ABC, abstractmethod

class RankingStrategy(ABC):
    @abstractmethod
    def sort_results(self, results: list[Result]) -> list[Result]:
        ...
```

15. Приклад Protocol

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Identifiable(Protocol):
    @property
    def id(self) -> int:
        ...

class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

16. Dunder methods (спеціальні методи)

У класах `RankingTable` та `Score` реалізовано методи: `__len__` (розмір таблиці), `__iter__` (ітерація в циклі for), `__contains__` (перевірка входження через 'in'), `__getitem__` (індексація table[0]), `__add__` (арифметика додавання балів), `__str__` та `__repr__`.

17. Generic class

```
class InMemoryRepository(Generic[TEntity]):
    def __init__(self) -> None:
        self._items: dict[int, TEntity] = {}

    def add(self, entity: TEntity) -> None:
        self._items[entity.id] = entity

    def get(self, entity_id: int) -> TEntity | None:
        return self._items.get(entity_id)
```

18. TypeVar

```
from typing import TypeVar
TEntity = TypeVar("TEntity", bound=Identifiable)
```

19. TypedDict

```
from typing import TypedDict, NotRequired

class ResultPayload(TypedDict):
    result_id: int
    athlete_id: int
    athlete_name: str
    country: str
    age: int
    sport_name: str
    competition_name: str
    location: str
    year: int
    score_value: float
    score_unit: str
    category: str
    notes: NotRequired[str]
```

20. Application service

```
class SportsAccountingService:
    def __init__(
        self,
        athlete_repo: InMemoryRepository[Athlete],
        result_repo: InMemoryRepository[Result],
        notifier: Notifier,
        exporter: ResultExporter,
    ) -> None:
        self._athlete_repo = athlete_repo
        self._result_repo = result_repo
        self._notifier = notifier
        self._exporter = exporter
```

21. Приклад Dependency Injection

```
athlete_repo = InMemoryRepository[Athlete]()
result_repo = InMemoryRepository[Result]()
notifier = ConsoleNotifier()
exporter = MarkdownExporter()

service = SportsAccountingService(
    athlete_repo=athlete_repo,
    result_repo=result_repo,
    notifier=notifier,
    exporter=exporter,
)
```

22. Пояснення принципів SOLID

- **SRP:** Клас `Score` відповідає виключно за значення та одиницю вимірювання, `RankingStrategy` — за алгоритм ранжування, `SportsAccountingService` — за бізнес-сценарії.
- **OCP:** Нові правила сортування додаються через нові класи-нащадки `RankingStrategy` без зміни коду існуючих класів.
- **LSP:** Підкласи `HigherScoreWinsStrategy` та `LowerTimeWinsStrategy` коректно замінюють базову абстракцію `RankingStrategy`.
- **ISP:** Протоколи розділені на вузькі незалежні контракти (`Identifiable`, `ScoredEntity`, `Notifier`, `ResultExporter`).
- **DIP:** `SportsAccountingService` залежить від абстракцій (`Notifier`, `ResultExporter`), а не від конкретних реалізацій поштових клієнтів чи файлових форматів.

23. Основний код програми

```
def main() -> None:
    service = build_sample_service()
    seed_demo_data(service)

    # Демонстрація Value Object
    s1 = Score(2.10, "m")
    s2 = Score(0.05, "m")
    print(f"Value Object Score: {s1} + {s2} = {s1 + s2}")

    # Пошук лідера
    best = service.find_best_result("Стрибки у висоту")
    if best:
        print(f"Лідер стрибків: {best.athlete.name} — {best.score}")

    # Контейнер RankingTable
    table = service.build_ranking_table("Стрибки у висоту")
    print(f"Елементів у таблиці: {len(table)}, Лідер: {table[0].athlete.name}")

    # Експорт звіту
    report = service.export_report("Стрибки у висоту")
    print(report)
```

24. Результат виконання програми (Screenshot execution)

```
================================================================================
  ЛАБОРАТОРНА РОБОТА №4: ПРОФЕСІЙНЕ ООП ТА СИСТЕМА ТИПІВ PYTHON
  Варіант 9: Система обліку спортивних результатів
================================================================================

[1] Ініціалізація сервісу через Dependency Injection:

[2] Реєстрація спортивних результатів (DTO Payload -> Domain Entity):
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Ярослава Магучіх — 2.10 m (Стрибк
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Ірина Геращенко — 2.00 m (Стрибки
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Юлія Левченко — 1.95 m (Стрибки у
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Ілля Ковтун — 15.65 pts (Спортивн
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Олег Верняєв — 15.30 pts (Спортив
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Михайло Романчук — 461.50 s (Плав
[NOTIFICATION] To admin@olympic.ua: Зареєстровано новий результат: Олександр Желтяков — 472.10 s (Пл

[3] Демонстрація Value Object Score (frozen=True, slots=True, __add__):
  2.10 m + 0.05 m = 2.15 m (Тип: Score)

[4] Пошук за видом спорту 'Стрибки у висоту':
  * Ярослава Магучіх       | 2.10 m | Diamond League
  * Ірина Геращенко        | 2.00 m | Олімпійські Ігри
  * Юлія Левченко          | 1.95 m | Чемпіонат України

[5] Найкращий результат у дисципліні 'Стрибки у висоту':
  Рекордсмен: Ярослава Магучіх — 2.10 m (Дорослі)

[6] Розрахунок статистики для 'Стрибки у висоту':
  Кількість спроб:     3
  Сумарна висота:      6.05 m
  Середня висота:      2.02 m
  Максимальна висота:  2.10 m
  Мінімальна висота:   1.95 m

[7] Контейнер RankingTable (Dunder методи: len=3, '101 in table'=True):
  Лідер таблиці (table[0]): Ярослава Магучіх з результатом 2.10 m

  Повна рейтингова таблиця (ітерація __iter__):
    1. Ярослава Магучіх       : 2.10 m (Дорослі)
    2. Ірина Геращенко        : 2.00 m (Дорослі)
    3. Юлія Левченко          : 1.95 m (Дорослі)

[8] Рейтинг для часової дисципліни 'Плавання 800м' (Стратегія: менший час кращий):
    1. Михайло Романчук       : 461.50 s (Дорослі)
    2. Олександр Желтяков     : 472.10 s (Юніори)

[9] Генерація та експорт звіту (ResultExporter Protocol):
[EXPORT] Generated Markdown report for 'Рейтинг дисципліни 'Стрибки у висоту''
# Рейтинг дисципліни 'Стрибки у висоту'

- 1. Ярослава Магучіх: 2.10 m (Дорослі)
- 2. Ірина Геращенко: 2.00 m (Дорослі)
- 3. Юлія Левченко: 1.95 m (Дорослі)
```

25. Результат перевірки mypy

```
$ python -m mypy src
Success: no issues found in 8 source files
```

26. Приклади знайдених type errors (експеримент розділу 71)

```
$ python -m mypy tests/type_errors_demo.py
tests\type_errors_demo.py:10: error: Incompatible types in assignment (expression has type "str", va
tests\type_errors_demo.py:14: error: Incompatible return value type (got "str", expected "int")  [re
tests\type_errors_demo.py:19: error: Argument 1 to "add" of "InMemoryRepository" has incompatible ty
Found 3 errors in 1 file (checked 1 source file)
```

27. Виправлений результат type checking

```
$ python -m mypy src
Success: no issues found in 8 source files
```

28. Відповіді на контрольні запитання

1. Що таке class?

Клас — це шаблон (креслення), який описує структуру атрибутів та поведінку (методи) створюваних на його основі об'єктів.

2. Що таке object?

Об'єкт — це сутність у пам'яті комп'ютера, яка має стан, поведінку та унікальний ідентифікатор.

3. Що таке instance?

Екземпляр конкретного класу, створений за його описом.

4. Для чого __init__()?

Спеціальний метод-ініціалізатор, що налаштовує початковий стан екземпляра під час його створення.

5. Що таке instance attribute?

Атрибут, який прив'язаний до конкретного екземпляра об'єкта (зберігається в self.__dict__).

6. Що таке class attribute?

Атрибут, визначений на рівні самого класу, спільний для всіх його екземплярів.

7. Що таке encapsulation?

Принцип ООП, за яким внутрішній стан об'єкта захищається від прямого неконтрольованого зовнішнього втручання та змінюється лише через публічні методи.

8. Для чого properties?

Декоратор @property дозволяє звертатися до методу як до атрибута з можливістю контролю читання, зміни (setter) та видалення (deleter).

9. Чим property відрізняється від public field?

Доступ до звичайного поля не перевіряється, а property дозволяє виконувати валідацію та обчислення при присвоєнні або зчитуванні.

10. Що таке inheritance?

Механізм мови, що дозволяє одному класу успадкувати структуру й поведінку іншого класу для повторного використання або розширення.

11. Що означає relation is-a?

Відношення спеціалізації між класами (наприклад, HigherScoreWinsStrategy is-a RankingStrategy).

12. Що таке composition?

Принцип побудови складних об'єктів шляхом об'єднання простіших і незалежних об'єктів як своїх полів.

13. Що означає relation has-a?

Відношення володіння (наприклад, Result has-a Athlete, Car has-an Engine).

14. Коли composition краща за inheritance?

У більшості випадків проектування архітектури, коли поведінка може динамічно змінюватися під час виконання та потрібно уникнути жорсткої зв'язаності монолітних ієрархій наслідування.

15. Для чого super()?

Для виклику реалізації методів батьківського класу згідно з порядком MRO (Method Resolution Order).

16. Що таке polymorphism?

Здатність різнорідних об'єктів реалізовувати однаковий інтерфейс з власною специфічною поведінкою.

17. Що таке duck typing?

Принцип динамічної типізації: якщо об'єкт реалізує необхідний набір методів та властивостей, він вважається придатним для використання незалежно від його положення в ієрархії наслідування.

18. Що таке ABC?

Абстрактний базовий клас (Abstract Base Class), який не дозволяє створювати власні екземпляри і зобов'язує підкласи реалізувати методи, помічені @abstractmethod.

19. Для чого abstractmethod?

Декоратор, що позначає метод в ABC як обов'язковий до реалізації в усіх конкретних нащадках.

20. Чи можна створити instance abstract class?

Ні, спроба інстанціювати клас, що має нереалізовані абстрактні методи, викликає помилку TypeError.

21. Що таке Protocol?

Механізм структурної типізації (PEP 544), що дозволяє описувати інтерфейси без потреби явного успадкування.

22. Чим Protocol відрізняється від ABC?

ABC вимагає явного наслідування (номінальна типізація), а Protocol перевіряє лише відповідність сигнатур методів (структурна типізація).

23. Що таке structural typing?

Система типів, де сумісність визначається формою об'єкта (наявністю полів і методів), а не ім'ям його батьківського класу.

24. Що таке nominal typing?

Система типів, де сумісність перевіряється за явним деревом наслідування та назвою типу.

25. Що таке dataclass?

Декоратор стандартної бібліотеки, що автоматично генерує шаблонні методи класів (__init__, __repr__, __eq__) на основі оголошених анотацій типів.

26. Які methods генерує dataclass?

За замовчуванням: __init__, __repr__, __eq__, а за налаштуваннями — __hash__, __lt__, __gt__ тощо.

27. Для чого default_factory?

Функція для безпечної ініціалізації змінюваних значень за замовчуванням (field(default_factory=list)), щоб уникнути спільного використання одного екземпляра списку всіма об'єктами.

28. Чому mutable default є проблемою?

Якщо список чи словник оголошено як значення за замовчуванням у конструкторі, він створюється один раз під час завантаження модуля і модифікується спільно всіма екземплярами.

29. Для чого __post_init__()?

Хук у dataclass, що автоматично викликається після стандартного __init__ для виконання додаткової валідації чи розрахунку похідних полів.

30. Що означає frozen=True?

Робить об'єкт датакласу незмінним (immutable): спроба змінити атрибут викликає виняток FrozenInstanceError.

31. Що означає slots=True?

Замінює стандартний динамічний словник __dict__ фіксованим кортежем слотів __slots__, що зменшує споживання оперативної пам'яті та прискорює доступ до полів.

32. Що таке dunder method?

Спеціальний магічний метод із подвійними підкресленнями (double underscore), що інтегрує користувацький клас у стандартну об'єктну модель мови Python.

33. Для чого __repr__()?

Повертає однозначне офіційне строкове представлення об'єкта для розробника та налагодження.

34. Для чого __str__()?

Повертає зручне для читання користувачем представлення об'єкта.

35. Для чого __len__()?

Викликається вбудованою функцією len() для повернення розміру контейнера.

36. Для чого __iter__()?

Повертає ітератор над елементами об'єкта для використання в циклі for чи функціях ітерації.

37. Для чого __contains__()?

Реалізує роботу оператора перевірки входження in.

38. Для чого __call__()?

Дозволяє викликати екземпляр класу як звичайну функцію: obj().

39. Що таке SOLID?

Акронім 5 основоположних принципів об'єктно-орієнтованого проектування: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

40. Що означає SRP?

Принцип єдиної відповідальності: клас повинен мати лише одну причину для зміни.

41. Що означає OCP?

Принцип відкритості/закритості: програмні сутності мають бути відкритими для розширення, але закритими для модифікації.

42. Що означає LSP?

Принцип підстановки Лісков: об'єкти підкласу мають бути взаємозамінними з об'єктами базового класу без порушення коректності програми.

43. Що означає ISP?

Принцип розділення інтерфейсу: клієнти не повинні залежати від методів, якими вони не користуються; краще багато вузьких інтерфейсів, ніж один широкий.

44. Що означає DIP?

Принцип інверсії залежностей: високорівневі модулі не повинні залежати від низькорівневих; обидва мають залежати від абстракцій.

45. Що таке dependency injection?

Патерн, коли залежності передаються об'єкту ззовні (через конструктор чи метод), а не створюються ним самостійно всередині.

46. Чому DI покращує testability?

Дозволяє легко заміняти важкі зовнішні ресурси (бази даних, мережеві сервіси) легкими mock-об'єктами в тестах.

47. Що таке type hint?

Анотація типів, що вказує очікуваний тип змінної, параметра чи поверненого функцією значення.

48. Чи перевіряє Python type hints автоматично в runtime?

Ні, інтерпретатор ігнорує type hints під час виконання; вони слугують документацією для IDE та аналізаторів типів.

49. Що таке static type checker?

Інструмент (наприклад, mypy, Pyright), який аналізує код до запуску без виконання програми та знаходить невідповідності типів.

50. Для чого mypy?

Найпопулярніший офіційний статичний типізатор для Python.

51. Для чого Pyright?

Швидкий статичний типізатор від Microsoft, що лежить в основі розширення Pylance у VS Code.

52. Що таке TypeVar?

Змінна типу в Python, яка дозволяє писати параметрично поліморфний (generic) код.

53. Що таке Generic?

Базовий клас для створення параметризованих типів, що працюють з різними конкретними типами без дублювання логіки.

54. Що означає Generic[T]?

Вказує, що клас параметризований змінною типу T.

55. Що таке bound TypeVar?

Обмеження типу зверху: TypeVar('T', bound=SomeClass), яке дозволяє використовувати лише SomeClass або його підтипи.

56. Що таке constrained TypeVar?

Обмеження типу чітким фіксованим списком типів: TypeVar('T', int, float).

57. Що таке TypedDict?

Механізм опису фіксованої структури ключів та їхніх типів для звичайних словників dict.

58. Чим TypedDict відрізняється від dataclass?

TypedDict під час виконання залишається звичайним словником без поведінки та методів, а dataclass створює повноцінний екземпляр користувацького класу.

59. Коли доцільно використовувати TypedDict?

Для валідації зовнішніх вхідних payload-структур (JSON, конфігураційні файли, відповіді API) перед конвертацією в доменні сутності.

60. Чому external payload і domain object краще розділяти?

Для ізоляції бізнес-логіки від змін у форматах зовнішніх API. Якщо зовнішній сервіс змінить назву полів JSON, це не вплине на внутрішні доменні сутності, а вимагатиме лише зміни коду перетворення (парсингу).

29. Висновок

Під час виконання лабораторної роботи № 4 було повністю опановано стандарти сучасного професійного об'єктно-орієнтованого проектування та системи статичної типізації Python:

1. **Доменна модель:** для індивідуального варіанта №9 («Система обліку спортивних результатів») спроектовано та реалізовано повний набір сутностей: `Athlete`, `Sport`, `Competition`, `Result` та `RankingTable`.
2. **Композиція та патерни проектування:** архітектура базується на перевазі композиції над наслідуванням. Реалізовано патерн **Strategy** через ієрархію `RankingStrategy` (ABC) для підтримки різних спортивних правил сортування.
3. **Value Objects та Dunder методи:** розроблено незмінний об'єкт значення `Score` (`frozen=True`, `slots=True`) з арифметикою та валідацією, а також реалізовано dunder-методи `__len__`, `__iter__`, `__contains__`, `__getitem__`.
4. **SOLID та інверсія залежностей:** застосовано структурні протоколи `Notifier` та `ResultExporter` у поєднанні з ручним Dependency Injection, що забезпечило слабку зв'язаність та високу тестованість системи.
5. **Статична типізація:** завдяки використанню `TypeVar`, `Generic`, `TypedDict` та суворому режиму аналізатора `mypy` забезпечено повну типову безпеку всього коду (`Success: no issues found in 8 source files`).

Створена об'єктна модель є надійною, розширюваною та відповідає найвищим стандартам сучасної корпоративної розробки на Python.